Modelado Relacional y Análisis de Datos de Partidos de Fútbol con MySQL y Node.js

Camilo Cuadros
Facultad de Ingeniería
Universidad de San Buenaventura
Bogotá, Colombia
cacuadrosm@academia.usbbog.edu.co

Juan Diego Sierra
Facultad de Ingeniería
Universidad de San Buenaventura
Bogotá, Colombia
jdsierram@academia.usbbog.edu.co

***Resumen*—Este trabajo presenta el diseño, la implementación y el análisis de una base de datos relacional en MySQL a partir de un archivo CSV con 4070 partidos de fútbol, correspondientes a las temporadas 2010/11 a 2020/21. Se justifica la selección de columnas, se transforma el modelo inicial hasta tercera forma normal, se implementa el esquema mediante DDL y se desarrolla un cargador en Node.js. La carga se verificó con los conteos de las cuatro tablas y el total de goles. Finalmente, se presentan cuatro consultas básicas y cuatro consultas analíticas; los resultados de referencia se calcularon a partir del CSV y se acompañan con capturas de las consultas ejecutadas desde el código.**

***Index Terms*—normalización, modelo relacional, MySQL, Node.js, SQL, análisis de datos, fútbol**

I. Introducción

Los datos tabulares pueden combinar en una sola fila información de varias entidades y repetir grupos de atributos. Un modelo relacional bien diseñado permite reducir redundancias y establecer reglas de integridad entre los datos [1]. En este proyecto se parte de un archivo CSV de estadísticas de partidos de fútbol y se transforma en un esquema normalizado en MySQL. Posteriormente, un programa en Node.js carga los registros y se ejecutan consultas que permiten verificar el funcionamiento del modelo y responder preguntas de análisis.

El documento describe el conjunto de datos, justifica las columnas conservadas y descartadas, presenta el modelo entidad-relación y explica las etapas de normalización. Después documenta el DDL, la arquitectura del cargador y las verificaciones de carga. Finalmente, presenta las ocho consultas SQL con resultados e interpretación.

II. Conjunto de datos y selección de información

II-A. Descripción del archivo

El archivo CSV suministrado para el laboratorio contiene 4070 filas y 33 columnas. Cada fila representa un partido con equipo local, equipo visitante, marcador y estadísticas de ambos equipos. El archivo comprende 11 temporadas, desde 2010/11 hasta 2020/21, y 37 equipos. Al tratarse del archivo descargable asignado para el ejercicio, el análisis se limita a describir y transformar su contenido.

La exploración inicial reportó ausencia de valores nulos, filas duplicadas y partidos de un equipo contra sí mismo. También se observó que `result` puede derivarse del marcador, que `sg_match_ft` coincide con la diferencia de goles y que la posesión de ambos equipos suma 100 en cada fila. Estos controles se deben conservar como pruebas reproducibles si se modifica el archivo de entrada.

Tabla I
Columnas conservadas del CSV

Columnas	Justificación
`season`, `date`	Identificación de temporada y fecha del partido.
`home_team`, `away_team`	Identifican los participantes y su condición de local o visitante.
`goal_home_ft`, `goal_away_ft`	Marcador final; permite calcular ganador, empate, puntos y goles totales.
`*_possession`	Permite estudiar la posesión en partidos ganados.
`*_shots`, `*_shots_on_target`	Permiten estudiar la actividad ofensiva y los tiros a puerta.
`*_yellow_cards`, `*_red_cards`	Permiten estudiar las tarjetas por equipo y temporada.

Tabla II
Columnas descartadas del CSV

Columnas	Justificación
`X`	Índice auxiliar generado al exportar el archivo; no representa una entidad del dominio.
`result`, `sg_match_ft`	Se derivan del marcador final, por lo que almacenarlos duplicaría información.
`*_clearances`, `*_corners`, `*_fouls_conceded`, `*_offsides`, `*_passes`, `*_tackles`, `*_touches`	No son requeridos por las ocho consultas planteadas. Excluirlos mantiene el esquema centrado en los objetivos del laboratorio.

II-B. Criterio de selección

La selección de columnas se hizo según las necesidades del modelo y las consultas solicitadas, no simplemente según la disponibilidad de datos.

III. Modelo entidad-relación

El esquema final contiene cuatro entidades:

- **season**: identifica una temporada mediante `season_id` y `label`.
- **team**: identifica un equipo mediante `team_id` y `name`.

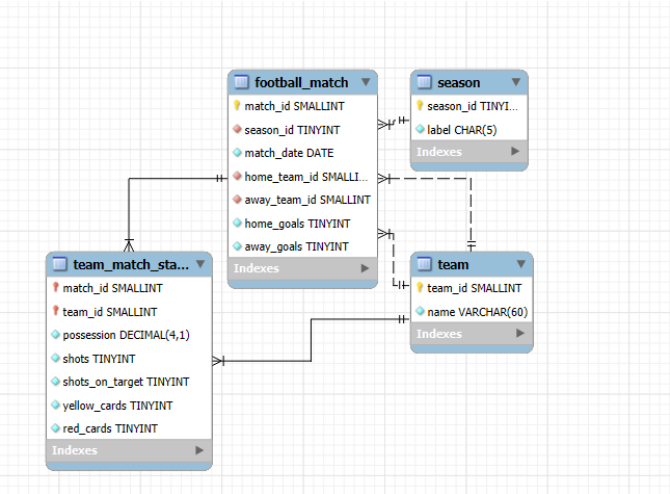


Figura 1. Diagrama entidad-relación final del esquema normalizado.

- **football_match**: registra temporada, fecha, equipos local y visitante, y goles de cada uno.
- **team_match_stats**: registra posesión, tiros, tiros a puerta y tarjetas de un equipo en un partido.

Una temporada puede tener muchos partidos y cada partido pertenece a una temporada (1:N). Un equipo puede aparecer en muchos partidos como local y en muchos como visitante; estas son dos relaciones distintas de 1:N. Un equipo también puede tener muchas filas en `team_match_stats`. Cada partido debe tener dos filas de estadísticas, una por participante; esta regla se espera del cargador y del conjunto de datos, aunque las claves foráneas y la clave primaria por sí solas no garantizan exactamente dos filas por partido.

En el modelo, cada partido tiene un identificador propio. Con ese identificador se puede conocer su temporada, fecha, equipos participantes y marcador. Para las estadísticas, en cambio, se necesita identificar tanto el partido como el equipo, porque un mismo equipo participa en muchos encuentros y sus datos cambian de un partido a otro. Por eso, la combinación de `match_id` y `team_id` identifica las estadísticas de un equipo en un partido específico. Además, cada identificador de equipo corresponde a un solo nombre, y cada identificador de temporada corresponde a una sola etiqueta.

La tabla `team_match_stats` solo debe contener equipos que realmente participaron en el partido correspondiente. Las claves foráneas comprueban que el partido y el equipo existan, pero no garantizan por sí solas que ese equipo sea el local o el visitante. Esa comprobación también la realiza el programa de carga antes de insertar los registros.

IV. Normalización

La normalización se aplicó por etapas, siguiendo los conceptos del modelo relacional y las formas normales [2]–[4]. Al comienzo, cada fila del CSV reunía en un mismo registro los datos del partido y las estadísticas del equipo local y del visitante. También incluía campos que podían calcularse a partir del marcador. El proceso consistió en organizar esa información para evitar repeticiones innecesarias y facilitar las consultas.

IV-A. Primera forma normal (1FN)

En este contexto, que un dato sea atómico significa que cada campo guarda un solo valor y no una lista de valores mezclados en la misma celda. Aunque el CSV cumple con esa idea para sus campos individuales, presenta las estadísticas en dos grupos de columnas: uno para el equipo local y otro para el visitante. Esto obliga a repetir la misma estructura de columnas para cada posición. Para organizar los datos de forma más uniforme, se separa la información general del encuentro en `football_match` y se crea `team_match_stats` para las estadísticas de cada participante. En esta última tabla, cada fila queda identificada por dos datos juntos: el partido al que pertenece y el equipo al que corresponden las estadísticas. De esta manera, las estadísticas del local y del visitante se guardan con las mismas columnas, sin duplicar el diseño para cada rol.

IV-B. Segunda forma normal (2FN)

En esta etapa no fue necesario dividir más las tablas. En `football_match`, cada registro se identifica con `match_id`, que por sí solo permite reconocer un partido. En `team_match_stats`, en cambio, se necesitan tanto el identificador del partido como el del equipo: la posesión, los tiros y las tarjetas no dependen únicamente del equipo ni únicamente del partido, sino de la participación de ese equipo en ese encuentro. Por eso, ambos identificadores se usan juntos para identificar cada registro de estadísticas.

IV-C. Tercera forma normal (3FN)

Se aplican tres decisiones. Primero, se excluyen `result` y `sg_match_ft`, pues pueden calcularse a partir de los goles y su almacenamiento duplicaría información. Segundo, los equipos y las temporadas se separan en las tablas `team` y `season`, evitando repetir sus nombres y etiquetas en cada partido. Tercero, se conserva explícitamente `season_id` en `football_match`: un encuentro aplazado puede disputarse en una fecha posterior y seguir perteneciendo a la temporada original. El esquema final reduce redundancia y permite aplicar integridad referencial.

V. Implementación del esquema

El archivo `sql/schema.sql` es la fuente del DDL utilizado en el repositorio. Crea la base `football_lab` y las tablas `season`, `team`, `football_match` y `team_match_stats`. Las claves foráneas conectan partidos con temporadas y equipos, mientras que la clave primaria compuesta de estadísticas evita registrar dos veces al mismo equipo en el mismo partido. Se usan restricciones `CHECK` para evitar partidos entre un equipo consigo mismo y valores de posesión fuera del intervalo de 0 a 100.

El DDL completo se mantiene en el archivo fuente del proyecto para evitar que el informe y el repositorio diverjan. La implementación se puede consultar en `sql/schema.sql`.

VI. Programa de carga en Node.js

VI-A. Decisión arquitectónica

El programa se ejecuta desde la terminal porque su tarea es procesar el archivo CSV y guardarlo en MySQL; no necesita atender solicitudes de una página web. Al separar algunas tareas en archivos, cada parte del código se ocupa de una labor concreta. Esto facilita ubicar errores y hacer cambios sin tener que modificar todo el proceso al mismo tiempo.

VI-B. Organización y responsabilidades

El código está distribuido en los siguientes archivos y carpetas:

- `src/index.js`: inicia el proceso y llama a los componentes necesarios en el orden correcto.
- `src/config/environment.js`: lee los datos de configuración, como los parámetros de conexión, desde las variables de entorno. Así no es necesario escribir las credenciales directamente en el código.
- `src/csv/matchCsvReader.js`: abre y procesa el CSV para entregar sus filas al resto del programa.
- `src/validation/matchRowValidator.js`: revisa que cada fila tenga los datos y valores requeridos. `rowValidationError.js` permite representar los errores de validación de forma clara.
- `src/database/connectionFactory.js`: crea la conexión con MySQL. `transaction.js`: controla la transacción, para confirmar los cambios si todo sale bien o revertirlos si ocurre un error.
- `src/repositories/`: contiene las operaciones que interactúan directamente con las tablas, como insertar temporadas, equipos, partidos y estadísticas, además de consultar los datos necesarios para verificar la carga.
- `src/services/matchLoaderService.js`: coordina el proceso principal de carga. `matchCollections.js` organiza los registros relacionados que se preparan para insertar.
- `src/verification/loadVerifier.js`: compara las cantidades y los goles cargados con los valores esperados.
- `src/errors/loadErrorDescriber.js`: transforma errores conocidos en mensajes que se puedan entender desde la terminal.

No es obligatorio que todos los proyectos tengan tantas carpetas y archivos; para un ejercicio pequeño, algunas tareas podrían reunirse en menos módulos. En este caso se documenta la estructura que ya tiene el repositorio. Aunque ocupa más archivos, permite distinguir con mayor facilidad la lectura, validación, acceso a la base de datos, carga y verificación.

El proyecto usa Node.js, `csv-parse` para procesar el CSV y `mysql2` para conectarse a MySQL. Si se detecta una fila inválida, el programa informa el número de línea y cancela la carga completa; si falla la inserción, la transacción permite revertir los cambios. Al finalizar, se verifican los conteos y el total de goles. El proceso termina con código distinto de cero si la carga o las comprobaciones fallan.

Tabla III
Verificación de los datos cargados en MySQL

Tabla / comprobación	Esperado	Cargado	Estado
`season`	11	11	OK
`team`	37	37	OK
`football\_match`	4070	4070	OK
`team\_match\_stats`	8140	8140	OK
Goles totales	11137	11137	OK

El punto de entrada ilustra el manejo explícito del resultado de la verificación:

```
runLoad()
  .then((verificationPassed) => {
    process.exitCode = verificationPassed ? 0 : 1;
  })
  .catch((error) => {
    console.error(`Error en la carga: ${describeLoadError(
     error)}`);
    process.exitCode = 1;
  });
```

Listing 1. Fragmento del punto de entrada del cargador.

VII. Verificación de la carga

Después de ejecutar `npm run load`, el programa comparó los valores esperados con los registros almacenados en MySQL. Los conteos coincidieron para las cuatro tablas y también coincidió el total de goles, por lo que la verificación terminó correctamente.

La salida de la terminal confirmó cada comprobación y finalizó con el mensaje *Verificación aprobada*. En particular, los 8140 registros de `team\_match\_stats` corresponden a dos registros por cada uno de los 4070 partidos.

Listing 2. Salida de la ejecución de npm run load.

```
> football-sql-lab@1.0.0 load
> node src/index.js

registros en season: esperado 11, cargado 11 [OK]
registros en team: esperado 37, cargado 37 [OK]
registros en football_match: esperado 4070, cargado 4070 [
     OK]
registros en team_match_stats: esperado 8140, cargado 8140
     [OK]
goles totales: esperado 11137, cargado 11137 [OK]
Verificación aprobada
```

VIII. Consultas básicas

Para cada consulta se presentan los resultados de referencia obtenidos a partir del CSV y, a continuación, una captura de la salida generada al ejecutar la consulta correspondiente del proyecto. Las capturas se cargarán en Overleaf con los nombres indicados en cada figura.

VIII-A. Filtrado con dos condiciones

Pregunta. ¿Qué partidos disputados en 2019 tuvieron seis o más goles en total?

```
SELECT match_id, match_date, home_goals, away_goals
FROM football_match
WHERE match_date BETWEEN '2019-01-01' AND '2019-12-31'
  AND home_goals + away_goals >= 6
ORDER BY match_date, match_id;
```

En base al CSV. Se identifican 18 partidos. Entre los primeros resultados por fecha están el 2 de enero (3–3), el 19 de enero (4–3, dos partidos) y el 29 de enero (4–2). **Interpretación.** La consulta combina un intervalo de fechas y una condición calculada sobre el marcador. Esto demuestra el uso de múltiples predicados en `WHERE`.

En base al código. La captura siguiente muestra la salida de la consulta ejecutada sobre la base de datos.

match_id	match_date	home_goals	away_goals
3383	2019-01-02	3	3
3130	2019-01-19	4	3
3135	2019-01-19	4	3
3374	2019-01-29	4	2
3273	2019-02-10	6	0
3043	2019-02-22	1	5
3400	2019-02-27	5	1
3277	2019-03-10	4	2
3121	2019-03-16	4	3
3200	2019-04-27	3	3
3287	2019-05-12	5	3
3487	2019-09-14	2	5
3536	2019-09-21	8	0
3606	2019-10-05	1	5
3427	2019-10-25	0	9
3559	2019-10-26	2	4
3677	2019-11-24	3	3
3792	2019-12-04	5	2
NULL	NULL	NULL	NULL

Figura 2. Salida de la consulta básica 1: filtrado con dos condiciones.

VIII-B. JOIN entre dos entidades

Pregunta. ¿Qué equipos locales lograron goleadas con seis o más goles de diferencia?

```
SELECT m.match_date, t.name AS home_team, m.home_goals, m.
    away_goals
FROM football_match m
JOIN team t ON t.team_id = m.home_team_id
WHERE m.home_goals >= m.away_goals + 6
ORDER BY m.match_date;
```

En base al CSV. Se identifican 17 partidos. Algunos ejemplos son Chelsea 6–0 West Bromwich Albion (14 de agosto de 2010), Manchester United 7–1 (27 de noviembre de 2010) y Manchester United 9–0 Southampton (2 de febrero de 2021). **Interpretación.** El JOIN permite recuperar el nombre legible del equipo local a partir de su clave foránea. El filtro compara la diferencia de goles, no solo el número de goles anotados.

En base al código. La captura siguiente muestra la salida de la consulta ejecutada sobre la base de datos.

match_date	home_team	home_goals	away_goals
2010-08-14	Chelsea	6	0
2010-08-21	Arsenal	6	0
2010-08-22	Newcastle United	6	0
2010-11-27	Manchester United	7	1
2011-08-28	Manchester United	8	2
2011-10-02	Fulham	6	0
2012-02-04	Arsenal	7	1
2012-12-23	Chelsea	8	0
2013-11-02	Manchester City	7	0
2013-11-24	Manchester City	6	0
2013-12-28	Hull City	6	0
2014-03-22	Chelsea	6	0
2014-10-18	Southampton	8	0
2015-05-10	Manchester City	6	0
2019-02-10	Manchester City	6	0
2019-09-21	Manchester City	8	0
2021-02-02	Manchester United	9	0

Figura 3. Salida de la consulta básica 2: JOIN entre dos entidades.

VIII-C. JOIN entre tres o más entidades

Pregunta. ¿Qué equipos recibieron tarjetas rojas en la temporada 20/21 desde febrero de 2021?

```
SELECT m.match_date, t.name AS team, st.red_cards
FROM football_match m
JOIN season s ON s.season_id = m.season_id
JOIN team_match_stats st ON st.match_id = m.match_id
JOIN team t ON t.team_id = st.team_id
WHERE s.label = '20/21'
  AND st.red_cards >= 1
  AND m.match_date >= '2021-02-01'
ORDER BY m.match_date, t.name;
```

En base al CSV. Se obtienen siete filas: Arsenal y Southampton el 2 de febrero (dos tarjetas rojas cada uno); Newcastle United y West Ham United el 6 de febrero; West Bromwich Albion el 20 de febrero; Sheffield United el 3 de marzo; y Burnley el 6 de marzo. **Interpretación.** La consulta relaciona partidos, temporadas, estadísticas por equipo y nombres de equipos. La granularidad del resultado es una fila por equipo que recibió al menos una tarjeta roja en un partido.

En base al código. La captura siguiente muestra la salida de la consulta ejecutada sobre la base de datos.

match_date	team	red_cards
2021-02-02	Arsenal	2
2021-02-02	Southampton	2
2021-02-06	Newcastle United	1
2021-02-06	West Ham United	1
2021-02-20	West Bromwich Albion	1
2021-03-03	Sheffield United	1
2021-03-06	Burnley	1

Figura 4. Salida de la consulta básica 3: JOIN entre tres o más entidades.

VIII-D. Agregación con GROUP BY

Pregunta. ¿Cuántos partidos hay por temporada y cuál es el promedio de goles por partido?

```
SELECT s.label AS season,
       COUNT(*) AS matches_played,
       ROUND(AVG(m.home_goals + m.away_goals), 2) AS
    avg_goals_per_match
FROM football_match m
JOIN season s ON s.season_id = m.season_id
GROUP BY s.season_id, s.label
ORDER BY s.label;
```

En base al CSV. El resumen por temporada se presenta en la Tabla IV; el archivo contiene 271 partidos para 20/21 y 379 para 15/16. **Interpretación.** La consulta agrupa los partidos por temporada y calcula un promedio. Las temporadas con menos partidos deben compararse con cuidado frente a las temporadas completas.

En base al código. La captura siguiente muestra la salida de la consulta ejecutada sobre la base de datos.

season	matches_played	avg_goals_per_match
10/11	380	2.80
11/12	380	2.81
12/13	380	2.80
13/14	380	2.77
14/15	380	2.57
15/16	379	2.70
16/17	380	2.80
17/18	380	2.68
18/19	380	2.82
19/20	380	2.72
20/21	271	2.60

Figura 5. Salida de la consulta básica 4: agregación por temporada.

VIII-E. Puntos acumulados y eficiencia por equipo

Pregunta. Si se asignan tres puntos por victoria y uno por empate, ¿qué equipo suma más puntos y es también el que más partidos ha disputado?

```
WITH team_results AS (
  SELECT home_team_id AS team_id,
         CASE WHEN home_goals > away_goals THEN 3
              WHEN home_goals = away_goals THEN 1 ELSE 0 END
     AS points
  FROM football_match
  UNION ALL
  SELECT away_team_id,
         CASE WHEN away_goals > home_goals THEN 3
              WHEN away_goals = home_goals THEN 1 ELSE 0 END
  FROM football_match
)
SELECT t.name AS team,
       COUNT(*) AS matches_played,
       SUM(r.points) AS total_points,
       ROUND(1.0 * SUM(r.points) / COUNT(*), 2) AS
    points_per_match,
       RANK() OVER (ORDER BY SUM(r.points) DESC) AS
    rank_by_points,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS
    rank_by_matches
FROM team_results r
JOIN team t ON t.team_id = r.team_id
GROUP BY t.team_id, t.name
ORDER BY total_points DESC
LIMIT 10;
```

En base al CSV. Manchester City ocupa el primer lugar por puntos acumulados con 891 y comparte el mayor número de partidos jugados (408) con Manchester United. Los resultados de los diez primeros equipos se presentan en la Tabla V. **Interpretación.** La tasa de puntos por partido ayuda a comparar el rendimiento sin depender únicamente del volumen de encuentros. La tabla está ordenada por puntos, no por puntos por partido.

En base al código. La captura muestra los resultados obtenidos al ejecutar la consulta analítica.

team	matches_played	total_points	points_per_match	rank_by_points	rank_by_matches
Manchester City	408	891	2.18	1	1
Manchester United	408	794	1.95	2	1
Chelsea	407	777	1.91	3	3
Liverpool	407	764	1.88	4	3
Tottenham Hotspur	405	740	1.83	5	7
Arsenal	407	738	1.81	6	3
Everton	406	598	1.47	7	6
West Ham United	368	451	1.23	8	9
Newcastle United	369	437	1.18	9	8
Southampton	331	426	1.29	10	11

Figura 6. Resultados de la consulta analítica 1: puntos acumulados por equipo.

VIII-F. Posesión promedio en los partidos ganados

Pregunta. ¿Qué equipo tiene la mayor posesión promedio cuando gana, y ganar con menos del 50 % de posesión es raro o común?

```
WITH team_wins AS (
  SELECT st.team_id, st.possession
  FROM football_match m
  JOIN team_match_stats st ON st.match_id = m.match_id
  WHERE (st.team_id = m.home_team_id AND m.home_goals > m.
     away_goals)
     OR (st.team_id = m.away_team_id AND m.away_goals > m.
     home_goals)
)
SELECT t.name AS team,
       COUNT(*) AS wins,
       ROUND(AVG(w.possession), 1) AS avg_possession_in_wins
    ,
       ROUND(100.0 * SUM(CASE WHEN w.possession < 50 THEN 1
    ELSE 0 END)
             / COUNT(*), 1) AS pct_wins_below_50
FROM team_wins w
JOIN team t ON t.team_id = w.team_id
GROUP BY t.team_id, t.name
ORDER BY avg_possession_in_wins DESC
LIMIT 10;
```

En base al CSV. De 3070 victorias registradas, 1362 ocurrieron con posesión inferior al 50 %, equivalentes al 44,4 %. **Interpretación.** Manchester City presenta la posesión media más alta entre sus victorias (62,3 %). Aun así, cerca del 44,4 % de las victorias se logró con menos de la mitad de la posesión, por lo que ganar con menor posesión no es excepcional en el conjunto de datos. Leeds United tiene solo 11 victorias en el archivo, de modo que su promedio debe interpretarse con cautela.

En base al código. La captura muestra los resultados obtenidos al ejecutar la consulta analítica.

	team	wins	avg_possession_in_wins	pct_wins_below_50
▶	Manchester City	275	62.3	12.7
	Leeds United	11	59.0	18.2
	Arsenal	215	58.9	20.5
	Liverpool	223	58.6	19.3
	Chelsea	229	57.5	21.0
	Tottenham Hotspur	216	56.9	24.1
	Manchester United	232	56.4	23.3
	Wigan Athletic	29	52.9	37.9
	Swansea City	82	51.8	43.9
	Southampton	112	51.3	42.0

Figura 7. Resultados de la consulta analítica 2: posesión en partidos ganados.

VIII-G. Diferencia entre tiros a puerta y goles

Pregunta. ¿Qué equipo tuvo, en una misma temporada, la mayor diferencia entre tiros a puerta y goles anotados?

```
WITH team_season_totals AS (
  SELECT s.label AS season, t.name AS team,
         SUM(st.shots_on_target) AS shots_on_target,
         SUM(CASE WHEN st.team_id = m.home_team_id
                  THEN m.home_goals ELSE m.away_goals END)
    AS goals_scored
  FROM football_match m
  JOIN season s ON s.season_id = m.season_id
  JOIN team_match_stats st ON st.match_id = m.match_id
  JOIN team t ON t.team_id = st.team_id
  GROUP BY s.season_id, s.label, t.team_id, t.name
)
SELECT season, team, shots_on_target, goals_scored,
       shots_on_target - goals_scored AS shots_without_goal,
       ROUND(100.0 * goals_scored / shots_on_target, 1) AS
    conversion_pct
FROM team_season_totals
ORDER BY shots_without_goal DESC
LIMIT 10;
```

En base al CSV. Tottenham Hotspur en 2015/16 registra la mayor diferencia del archivo: 252 tiros a puerta frente a 69 goles, una diferencia de 183. Los primeros resultados aparecen en la Tabla VII. **Interpretación.** Esta diferencia no equivale a tiros totales fallados, sino a tiros a puerta menos goles. La conversión es una métrica descriptiva agregada. La temporada 2015/16 contiene 379 partidos en el CSV, por lo que conviene advertir esa particularidad al comparar totales.

En base al código. La captura muestra los resultados obtenidos al ejecutar la consulta analítica.

	season	team	shots_on_target	goals_scored	shots_without_goal	conversion_pct
▶	15/16	Tottenham Hotspur	252	69	183	27.4
	10/11	Chelsea	244	69	175	28.3
	12/13	Tottenham Hotspur	241	66	175	27.4
	16/17	Tottenham Hotspur	257	86	171	33.5
	10/11	Arsenal	238	72	166	30.3
	18/19	Manchester City	260	95	165	36.5
	12/13	Manchester City	231	66	165	28.6
	11/12	Tottenham Hotspur	230	66	164	28.7
	19/20	Manchester City	266	102	164	38.3
	11/12	Arsenal	236	74	162	31.4

Figura 8. Resultados de la consulta analítica 3: tiros a puerta y goles.

VIII-H. Temporada con más tarjetas rojas

Pregunta. ¿En qué temporada hubo más tarjetas rojas, sumando ambos equipos, y coincide con la temporada con más goles?

```
SELECT s.label AS season,
       COUNT(*) AS matches_played,
       SUM(r.red_cards) AS total_red_cards,
       SUM(m.home_goals + m.away_goals) AS total_goals,
       ROUND(1.0 * SUM(r.red_cards) / COUNT(*), 3) AS
    red_cards_per_match,
       ROUND(1.0 * SUM(m.home_goals + m.away_goals) / COUNT
    (*), 2) AS goals_per_match
FROM football_match m
JOIN season s ON s.season_id = m.season_id
JOIN (
  SELECT match_id, SUM(red_cards) AS red_cards
  FROM team_match_stats
  GROUP BY match_id
) r ON r.match_id = m.match_id
GROUP BY s.season_id, s.label
ORDER BY total_red_cards DESC;
```

En base al CSV. La temporada 2014/15 tuvo más tarjetas rojas (71), mientras que 2018/19 registró el mayor total de goles (1072). Los resultados por temporada se resumen en la Tabla VIII. **Interpretación.** Las temporadas con más tarjetas rojas y más goles no coinciden. Los promedios por partido permiten comparar temporadas de distinto tamaño; en este archivo, 2020/21 contiene menos partidos que una temporada completa.

En base al código. La captura muestra los resultados obtenidos al ejecutar la consulta analítica.

	season	matches_played	total_red_cards	total_goals	red_cards_per_match	goals_per_match
▶	14/15	380	71	975	0.187	2.57
	11/12	380	66	1066	0.174	2.81
	10/11	380	65	1063	0.171	2.80
	15/16	379	59	1025	0.156	2.70
	13/14	380	53	1052	0.139	2.77
	12/13	380	52	1063	0.137	2.80
	19/20	380	48	1034	0.126	2.72
	18/19	380	47	1072	0.124	2.82
	16/17	380	41	1064	0.108	2.80
	17/18	380	39	1018	0.103	2.68
	20/21	271	35	705	0.129	2.60

Figura 9. Resultados de la consulta analítica 4: tarjetas rojas y goles por temporada.

VIII-I. Tabla de referencia calculada sobre el CSV

La tabla resume el número de partidos y el promedio de goles por temporada calculados directamente a partir del CSV.

IX. Consultas de análisis

IX-A. Tablas de referencia calculadas sobre el CSV

Las siguientes tablas reúnen los resultados tabulares calculados a partir del CSV. Se presentan al final de las consultas analíticas para mantener juntas las cifras de referencia y evitar que las tablas interrumpan la explicación de cada consulta.

X. Conclusiones

El desarrollo permitió pasar de un CSV que reunía en cada fila el marcador y las estadísticas de los dos equipos a un modelo relacional con cuatro tablas: temporadas, equipos, partidos y estadísticas por equipo-partido. Esta organización evita repetir los nombres de los equipos y las temporadas en cada registro, y permite relacionar los datos mediante claves primarias y foráneas. La normalización también permitió retirar campos que se pueden calcular a partir del marcador

Tabla IV
PARTIDOS Y PROMEDIO DE GOLES POR TEMPORADA (REFERENCIA CSV)

Temporada	Partidos	Goles/partido
10/11	380	2.80
11/12	380	2.81
12/13	380	2.80
13/14	380	2.77
14/15	380	2.57
15/16	379	2.70
16/17	380	2.80
17/18	380	2.68
18/19	380	2.82
19/20	380	2.72
20/21	271	2.60

Tabla V
PRIMEROS EQUIPOS POR PUNTOS ACUMULADOS (REFERENCIA CSV)

Equipo	PJ	Puntos	Puntos/PJ
Manchester City	408	891	2.18
Manchester United	408	794	1.95
Chelsea	407	777	1.91
Liverpool	407	764	1.88
Tottenham Hotspur	405	740	1.83
Arsenal	407	738	1.81
Everton	406	598	1.47
West Ham United	368	451	1.23
Newcastle United	369	437	1.18
Southampton	331	426	1.29

Tabla VI
POSESIÓN EN VICTORIAS, PRIMEROS DIEZ POR PROMEDIO (REFERENCIA CSV)

Equipo	Victorias	Posesión media (%)	Victorias con menos de 50 % (%)
Manchester City	275	62.3	12.7
Leeds United	11	59.0	18.2
Arsenal	215	58.9	20.5
Liverpool	223	58.6	19.3
Chelsea	229	57.5	21.0
Tottenham Hotspur	216	56.9	24.1
Manchester United	232	56.4	23.3
Wigan Athletic	29	52.9	37.9
Swansea City	82	51.8	43.9
Southampton	112	51.3	42.0

y almacenar las estadísticas de ambos participantes bajo una misma estructura.

La implementación en `schema.sql` establece las tablas y

Tabla VII
MAYOR DIFERENCIA ENTRE TIROS A PUERTA Y GOLES (REFERENCIA CSV)

Temporada	Equipo	Tiros a puerta	Goles	Diferencia	Conversión (%)
15/16	Tottenham Hotspur	252	69	183	27.4
10/11	Chelsea	244	69	175	28.3
12/13	Tottenham Hotspur	241	66	175	27.4
16/17	Tottenham Hotspur	257	86	171	33.5
10/11	Arsenal	238	72	166	30.3
18/19	Manchester City	260	95	165	36.5
12/13	Manchester City	231	66	165	28.6
11/12	Tottenham Hotspur	230	66	164	28.7
19/20	Manchester City	266	102	164	38.3
11/12	Arsenal	236	74	162	31.4

Tabla VIII
TARJETAS ROJAS Y GOLES POR TEMPORADA (REFERENCIA CSV)

Temporada	PJ	Rojas	Goles	Rojas/PJ	Goles/PJ
14/15	380	71	975	0.187	2.57
11/12	380	66	1066	0.174	2.81
10/11	380	65	1063	0.171	2.80
15/16	379	59	1025	0.156	2.70
13/14	380	53	1052	0.139	2.77
12/13	380	52	1063	0.137	2.80
19/20	380	48	1034	0.126	2.72
18/19	380	47	1072	0.124	2.82
16/17	380	41	1064	0.108	2.80
17/18	380	39	1018	0.103	2.68
20/21	271	35	705	0.129	2.60

sus relaciones, mientras que el programa en Node.js se encarga de leer el CSV, validar los registros, insertarlos y comprobar el resultado. La ejecución de `npm run load` terminó con la verificación aprobada: se cargaron 11 temporadas, 37 equipos, 4070 partidos y 8140 registros de estadísticas, y el total de goles coincidió con los 11137 esperados. Esto confirma que los datos cargados cumplen con los conteos establecidos por el proyecto.

Las consultas básicas muestran cómo filtrar registros con varias condiciones, relacionar tablas mediante `JOIN` y resumir partidos por temporada. Las consultas analíticas permiten comparar puntos acumulados, posesión en victorias, diferencia entre tiros a puerta y goles, y tarjetas rojas por temporada. En los cálculos de referencia sobre el CSV, Manchester City suma 891 puntos; el 44,4 % de las victorias se produjo con menos del 50 % de posesión; Tottenham Hotspur presenta en 2015/16 una diferencia de 183 entre tiros a puerta y goles; y la temporada 2014/15 registra más tarjetas rojas, mientras que 2018/19 tiene el mayor total de goles. En conjunto, el trabajo demuestra cómo un diseño relacional bien organizado y consultas SQL permiten convertir datos tabulares en resultados útiles para el análisis.

REFERENCIAS

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, Courant Computer Science Symposia, vol. 6, Prentice-Hall, 1972, pp. 33–64.

[3] W. Kent, “A simple guide to five normal forms in relational database theory,” *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, 1983.
[4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
[5] Oracle Corporation, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. Accedido: octubre de 2026.
[6] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/docs/latest/api/. Accedido: octubre de 2026.
[7] “MySQL2 documentation.” [Online]. Available: https://sidorares.github.io/node-mysql2/docs. Accedido: octubre de 2026.
[8] “CSV Parse documentation.” [Online]. Available: https://csv.js.org/parse/. Accedido: octubre de 2026.